

EDIM 설치·배포 매뉴얼

EDIM Tool System — NOVA Solution · 저장소 fankh/external-projects (edim-ai-blueprint/docs)

EDIM 설치·배포 매뉴얼

Self-managed 단일 서버 구성의 정식 설치·배포 절차. 본 절차는 개발 서버(edim.seekerslab.com, Ubuntu 24.04)에 실제 구축·검증된 구성을 정보으로 기록하며, §8의 운영 확장 구성(HA·큐·모니터링)은 본사업 이관 시 적용한다. 일상 운영·장애 대응은 관리자 가이드 참조.

항목	내용
문서 버전	v0.1
작성일	2026-07-11
대상	Self-managed 서버 설치 담당자 (Linux·Docker 기본 지식 전제)
검증 환경	Ubuntu 24.04 (16C/31GB) · Docker Compose · nginx · Let's Encrypt
관련 문서	컴포넌트 정의서 §8 · 관리자 가이드 · 보안관리계획서 §7

1. 구성 개요

단일 서버에 nginx(TLS 종단·정적·프록시) + Docker 컨테이너 4종(backend·postgres·minio·jenkins)을 배치한다.

구성 요소	배치	바인딩	역할
nginx (호스트)	apt 패키지	:80/:443	TLS 종단, 정적 서빙(/var/www/edim), /api 프록시, Basic Auth
edim-backend	앱 저장소 compose	127.0.0.1:8000	FastAPI — /api (프로토타입) · /api/v1 (업무 앱)
edim-postgres	인프라 compose	127.0.0.1:5432	PostgreSQL 16, db=edim (54테이블)
minio	인프라 compose	127.0.0.1:9000/9001	파일 스토리지 (버킷 edim — 도면·산출물)
jenkins (선택)	인프라 compose	127.0.0.1:8080/50000	CI — 자동 배포는 systemd 타이머가 기본

- 정적 자산: 프로토타입 SPA(/var/www/edim/) · 업무 앱(/var/www/edim/edim-static/) · 화면설계(design/) · 문서 포털(docs/)
- 내부 서비스는 전부 127.0.0.1 바인딩 — 외부 노출은 nginx 경유만 (보안 기준선: 보안관리계획서 §7.1)

2. 사전 요구사항

항목	요구
OS	Ubuntu 24.04 LTS (또는 systemd 기반 동급)
자원	최소 4C/8GB/50GB — 권장 8C/16GB+ (AI-CAD 변환 워커 포함 시)
소프트웨어	Docker Engine + Compose plugin · nginx · git · rsync · certbot · Python 3.12+ (문서 도구용, 선택)
네트워크	인바운드: SSH(사용자 지정 포트)·80·443 만 개방 (ufw) — DNS A 레코드가 서버 IP 지정
계정	sudo 가능한 서비스 계정, SSH 키 인증(비밀번호 로그인 차단)

3. 설치 절차

3.1 인프라 컨테이너 (postgres-minio-jenkins)

```
mkdir -p ~/apps/infra && cd ~/apps/infra
# .env - 시크릿 (chmod 600, 커밋 금지): POSTGRES_PASSWORD·MINIO_ROOT_USER/PASSWORD
# docker-compose.yml - postgres:16(edim-postgres, 127.0.0.1:5432, db=edim)
# minio(127.0.0.1:9000/9001) · jenkins-lts(127.0.0.1:8080, 선택)
docker compose up -d
docker compose ps          # 3종 healthy 확인
```

- MinIO 콘솔(/minio/ui/ 프록시 경유)에서 버킷 `edim` 생성 (또는 백엔드 첫 기동 시 자동 생성 확인)
- 네트워크 이름 `infra_default` 가 앱 compose의 external network — 이름 변경 시 앱 compose도 수정

3.2 앱 저장소·백엔드

```
mkdir -p ~/apps && cd ~/apps
git clone <저장소 URL> external-projects
cd external-projects/edim-ai-blueprint
cp backend/.env.example backend/.env    # 아래 $4 환경 변수 기입 (chmod 600)
docker compose up -d --build            # edim-backend → 127.0.0.1:8000
curl -s http://127.0.0.1:8000/api/v1/health # {"status":"ok","db":true} 확인
```

- DB 스키마·시드는 백엔드 기동 시 **역동 자동 실행** (테넌트 존재 시 버전별 증분만) — 수동 DDL 적용이 필요하면 `docs/ddl/edim_schema.sql`
- 초기 계정: 시드가 생성하는 `edim` (ADMIN) — **설치 직후 비밀번호 변경**

3.3 프론트엔드 정적 빌드·배치

```
# 업무 앱 (edim-web) - dist 가 저장소에 커밋되어 있으면 rsync 만
sudo mkdir -p /var/www/edim
sudo rsync -a --delete edim-web/dist/ /var/www/edim/edim-static/
# 프로토타입 SPA (선택)
docker run --rm -v ./frontend:/app -w /app node:20-alpine sh -c "npm ci && npm run build"
sudo rsync -a --delete frontend/dist/ /var/www/edim/
# 문서 포털·화면설계 (선택)
sudo rsync -a --delete docs/ /var/www/edim/docs/files/
sudo cp docs/portal.html /var/www/edim/docs/index.html
sudo chown -R www-data:www-data /var/www/edim
```

3.4 nginx-TLS

```
sudo htpasswd -c /etc/nginx/.edim_htpasswd edim    # 문서·프로토타입 경로 Basic Auth
# 사이트 설정 요지:
# / : /var/www/edim (Basic Auth)
# /docs/ /design/ : 정적 (Basic Auth)
# /cpq /plm /code /erp /toolbox /common /edim-static/ : SPA fallback (auth_basic off - 앱 자체 로그인)
# /api/ /api/v1/ : proxy_pass http://127.0.0.1:8000 (auth_basic off)
# /jenkins/ /minio/ui/ : 프록시 (auth_basic off - 자체 로그인)
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d <도메인>                  # TLS 발급 - 자동 갱신 타이머 확인
```

3.5 자동 배포·백업 타이머

```
# /usr/local/bin/edim-autodeploy.sh - fetch→pull→docker build→정적/docs rsync
# edim-autodeploy.timer : 2분 주기 (git push 만으로 배포)
# edim-backup.sh / edim-backup.timer : 매일 03:20 pg_dump + MinIO tar, 보존 7일
sudo systemctl enable --now edim-autodeploy.timer edim-backup.timer
journalctl -u edim-autodeploy | grep "deploy done" # 첫 배포 확인
```

4. 환경 변수 (backend/.env — chmod 600, 커밋 금지)

변수	필수	내용
DATABASE_URL	●	postgresql://edim:<암호>@edim-postgres:5432/edim (infra_default 네트워크 경유)
MINIO_ENDPOINT / MINIO_ACCESS_KEY / MINIO_SECRET_KEY	●	파일 스토리지 — 미설정 시 파일 업/다운로드 503
ANTHROPIC_API_KEY	○	AI 기능(도면 생성·Macro/UI 초안) 활성화 — 미설정 시 샘플 모드
ANTHROPIC_MODEL_ID	○	기본 claude-opus-4-8 (UI 모델 선택이 요청별 덮어씀)
ODA_FILE_CONVERTER_PATH	○	DWG→DXF 변환기 경로 — 미설정 시 DWG 업로드 501 (DXF 무관)
EDIM_DEV_MODE	○	1 = 요구사항 접수(📄) 노출 — 운영 배포에서는 미설정

5. 설치 검증 체크리스트

#	확인	기대
1	GET /api/v1/health	{ "status": "ok", "db": true }
2	앱 로그인 (/cpq)	로그인 후 상태바 DB: EDIM-PRD (PG16) (MOCK 아님)
3	C-1 제품 선정 → 슬롯 변경	BOM 실시간 재전개 (KDP 1-21-13-15 예시)
4	EDIM Run F9	6단계 완료 → Folder 에 DXF·PDF·XLSX 산출물, 다운로드 정상
5	파일 업로드 (Folder)	업로드→다운로드 바이트 일치 (MinIO 연결)
6	TLS	https:// 접속·인증서 유효, 80→443 리다이렉트
7	백업	첫 edim-backup 실행 후 덤프 파일 존재
8	(선택) 라이브 스위트	PYTHONUTF8=1 py tests/live_all.py 전체 통과

6. 업그레이드·롤백

- 업그레이드: git push → autodeploy 타이머가 2분 내 반영 (빌드 실패 시 기존 컨테이너 유지). 수동: git pull && docker compose up -d --build + 정적 rsync
- DB 변경: 시드·마이그레이션은 멎음 — 기동 시 자동. 대규모 스키마 변경은 백업 선행 후 적용
- 롤백: git checkout <직전 태그/커밋> 후 재빌드 + 직전 백업 복원 (pg_restore / MinIO tar) — RPO 24h/RTO 4h 목표(보안관리계획서 §3)

7. 제거(언인스톨)

컨테이너 중지·삭제(docker compose down) → 타이머 비활성 → /var/www/edim · ~/apps 제거. 데이터 파기는 백업 포함 완전 삭제 여부를 고객 보안 규정에 따라 확인 후 수행.

8. 운영 확장 구성 (본사업 — 컴포넌트 정의서 §8)

단일 서버 구성은 파일럿·소규모 Self-managed 용이며, SaaS/대규모 운영은 다음으로 확장한다:

항목	확장
INF-01	PostgreSQL HA (복제·자동 페일오버) + PITR
INF-03/04	큐(Redis Streams/RabbitMQ)로 Run-AI·변환 워커 분리·수평 확장, Redis 캐시
INF-05	Vector Store (pgvector/OpenSearch) — AI RAG
INF-06	Kubernetes + CI/CD 파이프라인 (Jenkins/Actions) — 현행은 systemd 타이머
INF-07	Prometheus·Grafana·Loki 모니터링 — 현행은 journalctl·헬스 체크

변경 이력

버전	일자	내용
v0.1	2026-07-11	최초 작성 — 개발 서버 검증 구성을 Self-managed 정식 절차로 문서화, 운영 확장(§8) 구분